LINUX — Guia Completo Profissional

Para absorção pelo Libertus

---

1. O QUE É LINUX

Linux é um sistema operacional de código aberto baseado em Unix, criado por Linus Torvalds em 1991. É o sistema mais usado em servidores, cloud, IoT e supercomputadores do mundo. Android é baseado em Linux.

Distribuições principais:

- Ubuntu — mais popular para iniciantes e servidores
- Debian — estável, base do Ubuntu
- Arch Linux — minimalista, rolling release
- Fedora — inovação, patrocinada pela Red Hat
- CentOS/Rocky Linux — servidores enterprise
- Kali Linux — segurança e pentest

---

2. ESTRUTURA DE DIRETÓRIOS

```
/                   → raiz do sistema
├── bin/            → binários essenciais (ls, cp, mv)
├── boot/           → arquivos de inicialização e kernel
├── dev/            → dispositivos (disco, USB, etc)
├── etc/            → configurações do sistema
├── home/           → pastas dos usuários (/home/lorran)
├── lib/            → bibliotecas compartilhadas
├── media/          → dispositivos montados (USB, CD)
├── mnt/            → pontos de montagem temporários
├── opt/            → software opcional de terceiros
├── proc/           → processos e info do kernel (virtual)
├── root/           → home do superusuário
```

```
├── run/            → dados de runtime
├── sbin/           → binários de sistema (para root)
├── srv/            → dados de serviços
├── sys/            → interface com o kernel (virtual)
├── tmp/            → arquivos temporários
├── usr/            → programas e dados do usuário
│   ├── bin/        → executáveis de usuário
│   ├── lib/        → bibliotecas
│   └── share/      → dados compartilhados
└── var/            → dados variáveis (logs, cache, db)
    ├── log/        → logs do sistema
    └── www/        → arquivos web
```

---

3. COMANDOS ESSENCIAIS

Navegação

```
pwd                     # mostra diretório atual
ls                      # lista arquivos
ls -la                  # lista com detalhes e ocultos
ls -lh                  # tamanhos legíveis
cd /caminho             # muda de diretório
cd ~                    # vai para home
cd ..                   # volta um nível
cd -                    # volta ao diretório anterior
```

Arquivos e Diretórios

```
mkdir pasta             # cria diretório
mkdir -p a/b/c          # cria hierarquia completa
touch arquivo.txt       # cria arquivo vazio
cp origem destino       # copia arquivo
cp -r pasta/ destino/   # copia diretório recursivo
mv origem destino       # move ou renomeia
rm arquivo              # remove arquivo
rm -rf pasta/           # remove diretório (CUIDADO)
ln -s alvo link         # cria link simbólico
```

Visualização

```
cat arquivo             # mostra conteúdo completo
```

```
less arquivo            # mostra paginado (q para sair)
head -n 20 arquivo      # primeiras 20 linhas
tail -n 20 arquivo      # últimas 20 linhas
tail -f log.txt         # segue arquivo em tempo real
wc -l arquivo           # conta linhas
wc -w arquivo           # conta palavras
```

Busca

```
find / -name "*.py"             # busca por nome
find . -type f -size +10M       # arquivos > 10MB
find . -mtime -7                # modificados nos últimos 7 dias
grep "texto" arquivo            # busca texto em arquivo
grep -r "texto" pasta/          # busca recursiva
grep -i "texto" arquivo         # ignora maiúsculas
grep -n "texto" arquivo         # mostra número da linha
locate arquivo                  # busca rápida (usa índice)
which python3                   # localiza executável no PATH
```

Permissões

```
ls -l                           # ver permissões
chmod 755 arquivo               # rwxr-xr-x
chmod +x script.sh              # torna executável
chmod -R 644 pasta/             # recursivo
chown usuario:grupo arquivo     # muda dono
chown -R lorran:lorran pasta/   # recursivo

# Permissões em octal:
# 7 = rwx (leitura + escrita + execução)
# 6 = rw- (leitura + escrita)
# 5 = r-x (leitura + execução)
# 4 = r-- (somente leitura)
# 0 = --- (nenhuma)

# Exemplo: chmod 755
# 7 = dono pode tudo
# 5 = grupo pode ler e executar
# 5 = outros podem ler e executar
```

Processos

```
ps aux                          # lista todos os processos
```

```
top                             # monitor de processos em tempo real
htop                            # monitor melhorado (instalar)
kill PID                        # encerra processo pelo ID
kill -9 PID                     # força encerramento
killall nome                    # mata por nome
pkill -f "python"               # mata por padrão
jobs                            # lista processos em background
bg                              # coloca processo em background
fg                              # traz processo para foreground
nohup comando &                 # executa sem travar ao fechar terminal
comando &                       # executa em background
```

Disco e Memória

```
df -h                           # espaço em disco
du -sh pasta/                   # tamanho de pasta
du -sh *                        # tamanho de cada item
free -h                         # memória RAM
lsblk                           # dispositivos de bloco
fdisk -l                        # partições (root)
mount /dev/sdb1 /mnt/usb        # monta dispositivo
umount /mnt/usb                 # desmonta
```

Rede

```
ip a                            # interfaces de rede e IPs
ip route                        # tabela de rotas
ping google.com                 # testa conectividade
curl https://site.com           # faz requisição HTTP
wget https://site.com/arquivo   # baixa arquivo
ss -tuln                        # portas abertas
netstat -tuln                   # portas (alternativa)
nmap 192.168.1.0/24             # escaneia rede local
ssh usuario@servidor            # acesso remoto
scp arquivo usuario@srv:/destino # copia via SSH
rsync -avz pasta/ usuario@srv:/ # sincroniza via SSH
```

---

4. GERENCIADORES DE PACOTES

APT (Ubuntu/Debian)

```
apt update                          # atualiza lista de pacotes
apt upgrade                         # atualiza pacotes instalados
apt install pacote                  # instala pacote
apt remove pacote                   # remove pacote
apt purge pacote                    # remove com configs
apt autoremove                      # remove dependências órfãs
apt search termo                    # busca pacotes
apt show pacote                     # info do pacote
dpkg -l                             # lista pacotes instalados
dpkg -i pacote.deb                  # instala .deb local
```

DNF/YUM (Fedora/CentOS)

```
dnf install pacote
dnf update
dnf remove pacote
dnf search termo
```

Pacman (Arch Linux)

```
pacman -Syu                         # atualiza sistema
pacman -S pacote                    # instala
pacman -R pacote                    # remove
pacman -Ss termo                    # busca
```

---

5. SHELL E SCRIPTS

Variáveis

```
NOME="Lorran"                       # define variável
echo $NOME                          # usa variável
echo "Olá, $NOME"                  # interpolação
export VARIAVEL="valor"             # exporta para subprocessos
env                                 # lista variáveis de ambiente
echo $PATH                          # PATH de executáveis
echo $HOME                          # pasta home
echo $USER                          # usuário atual
```

Scripts Bash

```bash
#!/bin/bash
# Shebang obrigatório na primeira linha

# Variáveis
NOME="Lorran"
IDADE=25

# Condicional
if [ $IDADE -ge 18 ]; then
    echo "Maior de idade"
elif [ $IDADE -gt 12 ]; then
    echo "Adolescente"
else
    echo "Criança"
fi

# Loop for
for i in 1 2 3 4 5; do
    echo "Número: $i"
done

# Loop while
CONTADOR=0
while [ $CONTADOR -lt 5 ]; do
    echo $CONTADOR
    CONTADOR=$((CONTADOR + 1))
done

# Funções
saudacao() {
    echo "Olá, $1!"    # $1 = primeiro argumento
}
saudacao "Lorran"

# Ler input
read -p "Seu nome: " NOME
echo "Olá, $NOME"

# Verificar se arquivo existe
if [ -f "arquivo.txt" ]; then
    echo "Arquivo existe"
fi

# Verificar se diretório existe
if [ -d "/pasta" ]; then
    echo "Diretório existe"
```

```
fi
```

Operadores de comparação

```
# Números:
-eq   # igual
-ne   # diferente
-gt   # maior que
-lt   # menor que
-ge   # maior ou igual
-le   # menor ou igual

# Strings:
=     # igual
!=    # diferente
-z    # vazia
-n    # não vazia

# Arquivos:
-f    # é arquivo
-d    # é diretório
-e    # existe
-r    # tem permissão de leitura
-w    # tem permissão de escrita
-x    # é executável
```

Pipes e Redirecionamento

```
comando > arquivo.txt            # redireciona saída (sobrescreve)
comando >> arquivo.txt           # redireciona (adiciona)
comando 2> erros.txt             # redireciona stderr
comando &> tudo.txt              # redireciona stdout e stderr
comando < entrada.txt            # redireciona entrada
comando1 | comando2              # pipe: saída de 1 vira entrada de 2

# Exemplos práticos:
ls -la | grep ".py"              # filtra arquivos .py
ps aux | grep python             # busca processos python
cat log.txt | sort | uniq        # ordena e remove duplicatas
history | grep "git"             # busca no histórico
cat /var/log/syslog | tail -100  # últimas 100 linhas do log
```

---

6. USUARIOS E GRUPOS

```
whoami                          # usuário atual
id                              # UID, GID e grupos
sudo comando                    # executa como root
su - usuario                    # muda de usuário
passwd                          # muda senha
passwd usuario                  # muda senha de outro (root)

useradd -m lorran               # cria usuário com home
useradd -m -s /bin/bash lorran  # com bash como shell
userdel -r lorran               # remove usuário e home
usermod -aG sudo lorran         # adiciona ao grupo sudo
usermod -aG docker lorran       # adiciona ao grupo docker

groupadd desenvolvedores        # cria grupo
groups lorran                   # grupos do usuário
cat /etc/passwd                 # lista usuários
cat /etc/group                  # lista grupos
```

---

7. SYSTEMD E SERVIÇOS

```
systemctl status nginx          # status do serviço
systemctl start nginx           # inicia serviço
systemctl stop nginx            # para serviço
systemctl restart nginx         # reinicia
systemctl reload nginx          # recarrega config
systemctl enable nginx          # ativa no boot
systemctl disable nginx         # desativa no boot
systemctl list-units --type=service  # lista serviços

journalctl -u nginx             # logs do serviço
journalctl -f                   # segue logs em tempo real
journalctl --since "1 hour ago"  # logs da última hora
journalctl -p err               # somente erros
```

Criar serviço systemd

```
# /etc/systemd/system/libertus.service
[Unit]
Description=Libertus AI System
After=network.target
```

```
[Service]
Type=simple
User=lorran
WorkingDirectory=/home/lorran/LIBERTUS/core
ExecStart=/usr/bin/python3 libertus.py
Restart=always
RestartSec=10
Environment=GROQ_API_KEY=sua_chave

[Install]
WantedBy=multi-user.target


systemctl daemon-reload          # recarrega configs
systemctl enable libertus        # ativa no boot
systemctl start libertus         # inicia agora
```

---

8. AMBIENTE PYTHON NO LINUX

```
python3 --version                # verifica versão
which python3                    # localiza executável
pip3 install pacote              # instala pacote
pip3 install -r requirements.txt # instala de arquivo

# Ambiente virtual (RECOMENDADO)
python3 -m venv .venv            # cria ambiente virtual
source .venv/bin/activate        # ativa
deactivate                       # desativa
pip freeze > requirements.txt   # exporta dependências

# pyenv (gerencia versões do Python)
curl https://pyenv.run | bash    # instala pyenv
pyenv install 3.12.0             # instala versão
pyenv global 3.12.0              # define versão global
pyenv local 3.11.0               # define versão local ao projeto
```

---

9. GIT NO LINUX

```
git init                         # inicializa repositório
git clone URL                    # clona repositório
```

```
git status                          # estado atual
git add arquivo                     # adiciona ao staging
git add .                           # adiciona tudo
git commit -m "mensagem"            # confirma mudanças
git push origin main                # envia para remoto
git pull                            # busca e merge
git branch                          # lista branches
git checkout -b nova-branch         # cria e muda de branch
git merge branch                    # merge
git log --oneline                   # histórico compacto
git diff                            # mostra diferenças
git stash                           # guarda mudanças temporariamente
git stash pop                       # recupera mudanças guardadas

# Configuração inicial
git config --global user.name "Lorran"
git config --global user.email "email@gmail.com"
git config --global core.editor "nano"
```

---

10. SSH — ACESSO REMOTO

```
# Gerar chave SSH
ssh-keygen -t ed25519 -C "lorran@email.com"
# Salva em ~/.ssh/id_ed25519 (privada) e ~/.ssh/id_ed25519.pub (pública)

# Copiar chave para servidor
ssh-copy-id usuario@servidor

# Conectar
ssh usuario@servidor
ssh -p 2222 usuario@servidor     # porta customizada
ssh -i ~/.ssh/chave usuario@srv  # chave específica

# Config SSH (~/.ssh/config)
Host meuservidor
    HostName 192.168.1.100
    User lorran
    IdentityFile ~/.ssh/id_ed25519
    Port 22

# Agora conecta com:
ssh meuservidor

# Túnel SSH (port forwarding)
```

```
ssh -L 8080:localhost:80 usuario@servidor   # local → remoto
ssh -R 9090:localhost:3000 usuario@servidor # remoto → local
```

---

11. DOCKER NO LINUX

```
# Instalação (Ubuntu)
apt install docker.io
systemctl enable --now docker
usermod -aG docker $USER         # sem precisar de sudo

# Comandos básicos
docker ps                        # containers rodando
docker ps -a                     # todos os containers
docker images                    # imagens locais
docker pull ubuntu               # baixa imagem
docker run -it ubuntu bash       # executa interativo
docker run -d nginx              # executa em background
docker run -p 8080:80 nginx      # mapeia porta
docker stop container_id         # para container
docker rm container_id           # remove container
docker rmi imagem                # remove imagem

# Docker Compose
docker-compose up -d             # sobe stack
docker-compose down              # derruba stack
docker-compose logs -f           # segue logs
docker-compose exec app bash     # entra no container

# Dockerfile básico
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "libertus.py"]
```

---

12. FERRAMENTAS DE PRODUTIVIDADE

tmux (múltiplos terminais)

```
tmux                              # inicia sessão
tmux new -s libertus              # sessão nomeada
tmux attach -t libertus           # reconecta à sessão
tmux ls                           # lista sessões

# Atalhos (Ctrl+B é o prefixo):
Ctrl+B, c    # nova janela
Ctrl+B, n    # próxima janela
Ctrl+B, %    # divide verticalmente
Ctrl+B, "    # divide horizontalmente
Ctrl+B, d    # detach (sessão continua rodando)
Ctrl+B, [    # modo scroll
```

nano (editor simples)

```
nano arquivo.txt
# Ctrl+O → salvar
# Ctrl+X → sair
# Ctrl+W → buscar
# Ctrl+K → corta linha
# Ctrl+U → cola linha
```

vim (editor profissional)

```
vim arquivo.txt
# Modos: Normal (padrão), Insert (i), Visual (v), Command (:)
# i → entra no modo insert
# Esc → volta ao modo normal
# :w → salva
# :q → sai
# :wq → salva e sai
# :q! → sai sem salvar
# dd → deleta linha
# yy → copia linha
# p → cola
# /texto → busca
# n → próximo resultado
# u → desfaz
# Ctrl+R → refaz
```

cron (agendamento)

```
crontab -e                        # edita tarefas agendadas
crontab -l                        # lista tarefas

# Formato: minuto hora dia mês dia_semana comando
# * = qualquer valor

# Exemplos:
0 9 * * 1-5 /home/lorran/backup.sh        # 9h em dias úteis
*/5 * * * * python3 /libertus/monitor.py  # a cada 5 minutos
0 0 * * * /scripts/limpeza.sh             # meia-noite todo dia
@reboot python3 /libertus/core/libertus.py # ao iniciar o sistema
```

---

13. SEGURANÇA BÁSICA

```
# Firewall (UFW - Ubuntu)
ufw enable                        # ativa firewall
ufw allow 22                      # permite SSH
ufw allow 80/tcp                  # permite HTTP
ufw allow 443/tcp                 # permite HTTPS
ufw deny 3306                     # bloqueia MySQL externo
ufw status                        # status e regras

# Verificar conexões suspeitas
ss -tuln                          # portas abertas
who                               # usuários logados
last                              # histórico de logins
auth.log                          # /var/log/auth.log — logins SSH

# Atualizar sistema (SEMPRE manter atualizado)
apt update && apt upgrade -y

# Desabilitar root via SSH
# Em /etc/ssh/sshd_config:
PermitRootLogin no
PasswordAuthentication no         # somente chaves SSH
```

---

14. MONITORAMENTO

```
# Sistema
top                               # monitor básico
```

```
htop                              # monitor avançado (colorido)
glances                           # monitor completo
iotop                             # I/O por processo
iftop                             # tráfego de rede

# Logs
tail -f /var/log/syslog           # log do sistema
tail -f /var/log/nginx/error.log # erros nginx
journalctl -f -u nome_servico     # logs de serviço

# Performance
vmstat 1                          # estatísticas do sistema
iostat 1                          # I/O de disco
mpstat 1                          # uso de CPU por core
```

---

15. COMANDOS ÚTEIS AVANÇADOS

```
# xargs — aplica comando a múltiplos itens
find . -name "*.pyc" | xargs rm  # remove todos .pyc
cat lista.txt | xargs wget       # baixa lista de URLs

# awk — processa texto
awk '{print $1}' arquivo         # primeira coluna
awk -F: '{print $1}' /etc/passwd # colunas separadas por :
ps aux | awk '{print $2, $11}'   # PID e comando

# sed — edita texto em stream
sed 's/antigo/novo/g' arquivo    # substituição global
sed -i 's/debug/info/g' *.py    # substitui in-place
sed '/^#/d' arquivo              # remove linhas de comentário

# sort e uniq
sort arquivo                     # ordena
sort -r arquivo                  # ordem reversa
sort -n numeros.txt              # ordena numericamente
uniq arquivo                     # remove duplicatas adjacentes
sort | uniq -c | sort -rn        # conta frequência

# tee — escreve em arquivo e stdout ao mesmo tempo
comando | tee arquivo.log

# tr — transforma caracteres
echo "hello" | tr 'a-z' 'A-Z'   # maiúsculas
echo "a,b,c" | tr ',' '\n'      # vírgula para nova linha
```

```
# diff — compara arquivos
diff arquivo1 arquivo2
diff -u arquivo1 arquivo2        # formato unificado (git style)

# tar — compactar
tar -czf backup.tar.gz pasta/    # compacta com gzip
tar -xzf backup.tar.gz           # descompacta
tar -tzf backup.tar.gz           # lista conteúdo

# Informações do sistema
uname -a                         # info do kernel
lscpu                            # info da CPU
lsmem                            # info da memória
lspci                            # dispositivos PCI
lsusb                            # dispositivos USB
hostname                         # nome do host
uptime                           # tempo ligado e carga
```

---

16. LIBERTUS NO LINUX

Instalação

```
# Clone ou copie o vault para o Linux
cp -r /media/usb/LIBERTUS ~/

# Instala dependências
cd ~/LIBERTUS/core
pip3 install httpx colorama anthropic

# Configura variáveis
export GROQ_API_KEY="sua_chave"
export ANTHROPIC_API_KEY="sua_chave"
export LIBERTUS_OBSIDIAN="/home/lorran/LIBERTUS"

# Torna permanente
echo 'export GROQ_API_KEY="sua_chave"' >> ~/.bashrc
echo 'export LIBERTUS_OBSIDIAN="/home/lorran/LIBERTUS"' >> ~/.bashrc
source ~/.bashrc

# Roda
python3 ~/LIBERTUS/core/libertus.py
```

Rodar 24/7 como serviço

```
# Cria o serviço
sudo nano /etc/systemd/system/libertus.service

# Conteúdo:
[Unit]
Description=Libertus v6.0
After=network.target

[Service]
Type=simple
User=lorran
WorkingDirectory=/home/lorran/LIBERTUS/core
ExecStart=/usr/bin/python3 libertus.py
Restart=always
Environment=GROQ_API_KEY=sua_chave_aqui
Environment=LIBERTUS_OBSIDIAN=/home/lorran/LIBERTUS

[Install]
WantedBy=multi-user.target

# Ativa
sudo systemctl daemon-reload
sudo systemctl enable libertus
sudo systemctl start libertus
sudo systemctl status libertus
```

Com Ollama pesado no Linux (quando tiver MacBook/PC bom)

```
# Instala Ollama
curl -fsSL https://ollama.ai/install.sh | sh

# Modelos recomendados por RAM:
# 8GB RAM:
ollama pull llama3:8b
ollama pull mistral:7b

# 16GB RAM:
ollama pull llama3:70b
ollama pull mixtral:8x7b

# 32GB+ RAM:
ollama pull llama3:70b-q8
```

Documento gerado para absorção pelo Libertus v6.0 Criado em: 13/04/2026